

BURP SUITE – WEB APPLICATION PENETRATION TESTING FRAMEWORK

*A SYSTEMATIC FRAMEWORK FOR IDENTIFYING AND MITIGATION
OWASP VULNERABILITIES*



Document Context: Technical Assessment Report

Prepared By

**TIRUMALASETTY MOHINI
TIRUMALASETTY JEEVAN KUMAR**

1.Core Tools:

- i. Kali Linux
- ii. Oracle VM Virtual Machine
- iii. Burp Suite Professional

2.Target Application:

- i. OWASP Juice-shop

3.Vulnerability Scope:

- i. SQL INJECTION
- ii. Cross Site Scripting
- iii. Brute Force Authentication

Date Issued: April 2026

ABSTRACT

Web applications have become an essential part of modern digital systems, providing services in areas such as banking, education, e-commerce, and communication. As the use of web applications continues to grow, ensuring their security has become a critical concern for organizations and developers. Many web applications handle sensitive user data such as personal information, login credentials, and financial details. If these applications contain security weaknesses, attackers may exploit them to gain unauthorized access, steal sensitive data, or disrupt system functionality.

This project focuses on the implementation of a web application penetration testing framework using Burp Suite to identify common security vulnerabilities in web applications. Penetration testing is a security assessment technique where testers simulate real-world attacks to discover weaknesses in a system before malicious attackers can exploit them.

The main objective of this project was to analyze how web applications process user input, identify potential vulnerabilities, and demonstrate how these weaknesses could be exploited in a controlled testing environment.

The testing environment for this project was created using Kali Linux running inside a virtual machine. Kali Linux provides a wide range of tools designed specifically for penetration testing and cybersecurity research. To safely perform security testing, an intentionally vulnerable web application called OWASP Juice Shop was used as the target system.

This application is developed by OWASP and is widely used for learning and practicing web security testing techniques. The application was deployed using Docker, which allowed the application to run inside a containerized environment without affecting the host system.

Burp Suite was used as the primary testing tool to intercept and analyze HTTP and HTTPS communication between the web browser and the target application. Burp Suite acts as an intercepting proxy that allows testers to capture, inspect, and modify requests sent from the browser to the web server.

This capability makes it possible to understand how the application processes user inputs and how attackers might manipulate requests to exploit vulnerabilities. Several types of web application attacks were performed during the penetration testing process.

One of the main vulnerabilities tested was SQL Injection. This attack occurs when an application fails to properly validate user input before including it in database queries. By injecting specially crafted SQL commands into input fields, attackers may be able to bypass authentication mechanisms or access sensitive information stored in the database.

Using Burp Suite, HTTP requests were intercepted and modified to test whether the application was vulnerable to SQL injection attacks. Another vulnerability tested in this project was Cross-Site Scripting (XSS). XSS occurs when an application allows malicious scripts to be injected into web pages that are viewed by other users.

These scripts can execute within the victim's browser and potentially steal session cookies, modify webpage content, or redirect users to malicious websites. By injecting script-based payloads into

input fields and analysing server responses, the project demonstrated how XSS vulnerabilities can be identified during penetration testing.

In addition to vulnerability testing, the project also examined authentication weaknesses in the application. A brute force attack was conducted to test the strength of the login mechanism by attempting multiple password combinations.

By capturing login requests and sending them to the Intruder module in Burp Suite, automated password testing was performed to determine whether the authentication system could be bypassed. Eventually, the credentials were identified and used to successfully log in as an administrator. The login request was further verified using the Repeater tool in Burp Suite to confirm that the authentication bypass was successful.

The results of the penetration testing process revealed several security weaknesses in the application, including improper input validation and insufficient protection against authentication attacks. These findings highlight the importance of implementing secure coding practices and performing regular security testing during the software development lifecycle.

Without proper security controls, web applications remain vulnerable to attacks that could compromise sensitive user data and system integrity.

Overall, this project demonstrates how a structured web application penetration testing framework can be used to identify and analyse security vulnerabilities in modern web applications. By using tools such as Burp Suite and testing against a controlled vulnerable environment, it is possible to simulate real-world attack scenarios and understand how security weaknesses can be exploited. The knowledge gained from this project contributes to a better understanding of web application security and highlights the importance of proactive security testing in protecting digital systems and user data.

In conclusion, the project successfully illustrates the role of penetration testing in identifying vulnerabilities and improving web application security. By combining practical testing techniques with proper security recommendations, the framework provides valuable insights into how organizations can detect potential threats and strengthen their defenses against cyber attacks. Future improvements to the framework may include integrating additional testing tools, automated vulnerability scanners, and advanced Burp Suite extensions to enhance the overall effectiveness of the penetration testing process.

Keywords: Web Application Security, Web Application Penetration Testing (WAPT), Burp Suite Framework, Kali Linux Environment, OWASP Juice Shop, SQL Injection, Cross-Site Scripting (XSS), Brute Force Attack, HTTP/HTTPS Traffic Interception, Vulnerability Assessment, Secure Coding Practices, Ethical Hacking.

INDEX

S. NO	LIST OF CONTENTS	PAGE NO
1	ABSTRACT	2-3
2	INTRODUCTION	5-6
3	LITERATURE REVIEW	7-9
4	METHODOLOGY	10
5	ENVIRONMENT SETUP	11-14
6	PENETRATION TESTING AND IMPLEMENTATION	15-21
7	RISKS AND ETHICAL CONSIDERATIONS	22
8	TABLE OF DISCOVERED VULNERABILITIES	23-24
9	CONCLUSION AND FUTURE SCOPE	25-26
10	REFERENCES AND APPENDIX	27

INTRODUCTION

Background of Web Applications and Web Security

With the rapid growth of the internet, web applications have become a major part of everyday digital activities. Many services such as online banking, shopping platforms, educational portals, and social media operate through web-based systems. Because these applications handle important user information including personal data, financial records, and login credentials, protecting them from cyber threats has become a major priority for organizations.

Web security focuses on protecting websites and web applications from unauthorized access, data breaches, and malicious attacks. Over time, cyber attackers have developed various techniques to exploit weaknesses in web applications.

Common threats include SQL Injection, Cross-Site Scripting (XSS), session hijacking, and authentication bypass. These vulnerabilities can allow attackers to steal sensitive information, manipulate data, or even gain full control over the application.

To address these risks, organizations perform **web application penetration testing**, which is a controlled security testing process designed to identify vulnerabilities before they can be exploited by real attackers.

During penetration testing, security professionals simulate attack techniques to evaluate how secure a web application is. This process helps organizations understand potential security weaknesses and improve their defences.

Various security tools and frameworks are used to perform web application penetration testing. One widely used tool is Burp Suite, developed by Port Swigger. Tools like Burp Suite allow testers to intercept web traffic, analyse requests and responses, and test how applications handle different types of input.

By carefully examining these interactions, testers can detect vulnerabilities and recommend appropriate security improvements. Conducting regular penetration testing helps organizations identify risks early, strengthen their security mechanisms, and protect sensitive information from cyber threats.

Project Objectives

The main goal of this project is to understand and demonstrate how web application security testing can be performed using Burp Suite. Web applications often handle sensitive data and user interactions, which makes them potential targets for cyberattacks. By using security testing tools and techniques, vulnerabilities can be identified and addressed before they are exploited by malicious attackers. One of the primary objectives of this project is to study the concepts of web application security and penetration testing. This includes understanding how web applications communicate with servers using HTTP and HTTPS protocols and how attackers may attempt to exploit weaknesses in these systems.

Another objective is to explore the different tools and features provided by Burp Suite. The project aims to analyse how components such as Proxy, Intruder, Repeater, and Scanner can be used to intercept web traffic, test application inputs, and detect security vulnerabilities.

The project also focuses on identifying common web application vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). Through practical testing and analysis, the project demonstrates how these vulnerabilities can be detected and how applications can be evaluated for potential security risks.

Another important objective is to understand the process of conducting penetration testing in a structured and ethical manner. This includes learning how to analyse requests and responses, perform controlled security tests, and document findings effectively. The project aims to highlight the importance of secure web application development by providing recommendations for improving application security. By identifying vulnerabilities and understanding their impact, organizations can take appropriate steps to strengthen their systems and protect sensitive information.

Scope

The scope of this project focuses on performing web application security testing using **Burp Suite** as the primary testing tool. The purpose of this project is to analyse and demonstrate how penetration testing techniques can be applied to identify potential vulnerabilities in a controlled web application environment. The testing activities are carried out only within a safe and authorized setup designed for learning and experimentation.

In this project, the security testing is conducted on a **deliberately vulnerable web application or a broken virtual machine (VM)** that is created specifically for practicing web security testing. Examples of such environments include intentionally vulnerable applications like DVWA (Damn Vulnerable Web Application) or OWASP Juice Shop. These platforms are commonly used for educational purposes because they contain known vulnerabilities that allow testers to safely practice penetration testing techniques.

The project mainly focuses on analyzing how web applications process user input and respond to requests. During the testing process, different types of vulnerabilities are examined, including input validation issues, authentication weaknesses, and common web attacks such as SQL Injection and Cross-Site Scripting (XSS). Burp Suite is used to intercept HTTP and HTTPS traffic, inspect requests and responses, and test how the application behaves when different inputs are provided.

However, the scope of this project is limited to **web application testing within the defined testing environment**. The project does not include testing real-world production websites, attacking external systems, or performing any unauthorized penetration testing activities. Network infrastructure testing, server exploitation, and denial-of-service attacks are also outside the scope of this study. By clearly defining these boundaries, the project ensures that all testing activities are conducted in a responsible and ethical manner while focusing specifically on learning and demonstrating web application penetration testing techniques.

LITERATURE REVIEW

BURP SUITE

Burp Suite works mainly as an intercepting proxy that sits between a user's web browser and the target web application. In simple terms, it acts like a middle layer that captures the communication happening between the client and the server. This allows security testers to observe the requests sent from the browser and the responses returned by the server.

Testers can also modify these messages before they reach their destination, which helps them study how the application reacts to different inputs. By doing this, penetration testers can uncover common web security issues such as SQL injection, cross-site scripting (XSS)penetration testers, bug bounty participants, and cybersecurity researchers across the world, weaknesses in authentication systems, and problems related to session handling.

Another important feature of Burp Suite is the collection of tools that are integrated into the platform to support different phases of security testing. For example, the Proxy tool allows testers to intercept and examine network traffic, while the Scanner can automatically search for possible vulnerabilities in the application.

The Intruder tool is useful for sending multiple customized requests to the server in order to test how the application handles unusual or unexpected input. In addition, the Repeater tool helps testers resend and modify requests manually to observe the application's behaviour more closely. Other utilities such as Decoder and Comparer assist in processing encoded information and comparing responses to detect differences.

With the help of these features, Burp Suite makes it easier for security professionals to evaluate the security of web applications in a structured way. It helps them detect weaknesses, understand potential risks, and suggest improvements to strengthen the system. Because of its flexibility and wide range of testing capabilities, Burp Suite has become a preferred tool for penetration testers, bug bounty participants, and cybersecurity researchers across the world.

Objectives of Using Burp Suite

The main objectives of using Burp Suite in web application penetration testing include:

- Identifying vulnerabilities in web applications
- Intercepting and analyzing HTTP/HTTPS traffic
- Testing authentication and session management mechanisms
- Detecting common web security vulnerabilities
- Improving the security posture of web applications

Burp Suite Architecture

Burp Suite functions as a **proxy server** that sits between the user's browser and the web application server. The architecture generally includes three main components:

- 1. Web Browser:** The browser sends HTTP/HTTPS requests to the web application.
- 2. Burp Suite Proxy:** Burp Suite intercepts the communication and allows testers to inspect or modify requests and responses.

3. Target Web Server: The server processes the request and returns the response to the client.

Key Tools in Burp Suite

Burp Suite provides several integrated tools that assist penetration testers in analyzing and identifying vulnerabilities in web applications. Each tool performs a specific role during the security testing process.

- **Proxy**

The Proxy tool acts as an intermediary between the user's browser and the web server. It captures the HTTP and HTTPS requests that are sent from the browser before they reach the server. This allows the tester to examine the request details and, if necessary, modify parameters, headers, or cookies before forwarding them. By doing this, testers can better understand how the web application processes user input and identify potential security weaknesses.

- **Scanner**

The Scanner is designed to automatically analyze web applications and search for known security vulnerabilities. It sends various requests to the application and observes how the server responds. Based on these responses, the tool can detect issues such as mis-configurations insecure input handling, or other common web security problems. This automated feature helps speed up the testing process by identifying possible vulnerabilities quickly.

- **Intruder**

The Intruder tool is used to perform automated testing by sending a large number of modified requests to the target application. Testers can use it to check how the system reacts to different inputs, including unusual or unexpected values. It is commonly used for activities such as password guessing, testing input fields for vulnerabilities, and performing fuzzing attacks to find weaknesses in application logic.

- **Repeater**

Repeater allows testers to manually resend HTTP requests to the server and analyze the responses in detail. This tool is especially useful when testers want to experiment with small modifications to a request and observe how the web application behaves. By repeating and adjusting requests, testers can better understand how specific inputs affect the application.

- **Decoder**

The Decoder tool helps testers work with encoded or encrypted data that may appear in web communication. It can transform encoded information into a readable format and also perform the reverse operation. For example, it can decode values encoded with Base64 or URL encoding so that testers can analyze the actual data being transmitted.

- **Comparer**

The Comparer tool is used to analyze the differences between two pieces of data. For example, testers can compare two server responses to determine how the application behaves when different inputs are provided. This helps identify subtle variations that may indicate a vulnerability or unexpected system behavior.

Burp Suite Comparison with Other Web Application Penetration Testing Tools

Web application penetration testing can be performed using several security tools. However, Burp Suite has become one of the most widely used platforms among penetration testers and cybersecurity professionals. Some popular alternatives include **OWASP ZAP**, **Nikto**, and **Nessus**. Each tool has different strengths and is used for specific purposes.

Burp Suite vs OWASP ZAP

- **OWASP ZAP** (Zed Attack Proxy) is an open-source web security scanner developed by the OWASP community. Like Burp Suite, it can intercept HTTP traffic and perform vulnerability scanning. However, ZAP focuses more on automated scanning and integration with development pipelines, while Burp Suite is often preferred for manual testing and detailed vulnerability analysis.
- **ZAP** is free and open-source, which makes it attractive for beginners and organizations with limited budgets. Burp Suite, on the other hand, provides more advanced testing tools and professional-grade features that are widely used in professional penetration testing environments.

Burp Suite vs Nikto

- **Nikto** is another tool used in web application security testing. It is mainly a command-line vulnerability scanner that checks web servers for outdated software, insecure configurations, and known vulnerabilities. Unlike Burp Suite, Nikto does not provide interactive traffic interception or manual testing capabilities. Because of this limitation, it is usually used as a supplementary scanning tool rather than a complete penetration testing framework.

Burp Suite vs Nessus

- **Nessus** is a vulnerability assessment tool that scans systems, networks, and web applications for security weaknesses. While Nessus is powerful for detecting known vulnerabilities across large systems, it is not specifically designed for detailed manual testing of web applications.

Why Burp Suite is Widely Used;

There are several reasons why Burp Suite is considered a standard tool for web application penetration testing:

- Provides both **manual and automated security testing capabilities**
- Allows detailed inspection and modification of **HTTP/HTTPS** requests
- Includes multiple integrated tools within a single platform
- Supports extensions and plugins for advanced testing
- Widely adopted by penetration testers and bug bounty researchers

Because of these capabilities, Burp Suite is commonly used in professional security testing environments, bug bounty programs, and cybersecurity research.

METHODOLOGY

This project follows the guidelines of the Penetration Testing Execution Standard (PTES) to perform web application penetration testing in a structured and systematic way.

1. Project Planning and Scope Setup

In the initial phase, the objectives and boundaries of the testing activity are defined. The target web application is selected and the testing environment is confirmed to be a controlled system designed for security practice, such as a vulnerable web application running on a virtual machine

2. Information Gathering

The next step involves collecting information about the target web application. Tools like Burp Suite are used to intercept HTTP requests and responses in order to understand how the application communicates with the server. This information helps identify possible entry points where vulnerabilities may exist.

3. Vulnerability Analysis

After understanding the application's structure, different testing techniques are applied to identify security weaknesses. The application inputs, login forms, and parameters are carefully analyzed to determine whether they are properly validated. During this phase, the tester looks for common web vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). By modifying requests through Burp Suite, it becomes possible to observe how the application responds to unusual or malicious input.

4. Exploitation Phase

In the exploitation phase, Several attack techniques were performed during the project:

- **Brute Force Attack:** Automated attempts were made to guess login credentials using different combinations of usernames and passwords.
- **SQL Injection Testing:** Input fields were tested with specially crafted SQL queries to determine whether the database queries were properly secured.
- **Cross-Site Scripting (XSS):** Script code was inserted into input fields to check whether the application improperly executes user-supplied scripts.

5. Post-Exploitation Verification

After obtaining administrative credentials, the results were verified to confirm that the exploitation was successful. The captured login request was analyzed and re-tested using the **Repeater** feature within Burp Suite. By resending the request through the Repeater tool, the tester confirmed that the credentials allowed consistent access to the administrator account.

6. Reporting and Documentation

The final phase involves documenting all findings discovered during the testing process. The identified vulnerabilities, attack methods used, and successful exploitation steps are recorded in detail. . By following the PTES methodology, The results demonstrate how structured testing methods and tools like Burp Suite can help identify critical vulnerabilities in web applications and improve their overall security.

ENVIRONMENT SETUP

Technical Stack

The testing environment for this project was set up using a controlled virtual laboratory to safely perform web application penetration testing. The main operating system used for security testing was **Kali Linux**, which is a specialized Linux distribution designed for cybersecurity professionals and penetration testers. **Kali Linux** provides a wide range of pre-installed security tools that assist in vulnerability analysis, network testing, and ethical hacking activities.

To create a safe and isolated testing environment, Kali Linux was installed inside a virtual machine using **Oracle VM Virtual Machine**. A virtual machine allows multiple operating systems to run on a single physical computer while keeping them separated from the host system. This setup ensures that penetration testing activities can be performed without affecting the main operating system or external networks.

Within the Kali Linux environment, Burp Suite was used as the primary tool for web application penetration testing. Burp Suite enabled the interception and analysis of **HTTP** and **HTTPS** traffic between the browser and the target web application. The tool was configured as a proxy so that all browser requests passed through Burp Suite before reaching the server, allowing detailed inspection and modification of requests and responses.

For testing purposes, a deliberately vulnerable web application environment was used. This type of environment is designed specifically for security learning and practice, allowing testers to safely identify and exploit vulnerabilities such as authentication flaws, input validation issues, and common web attacks like **SQL Injection** and **Cross-Site Scripting (XSS)**.

The overall technical stack used in this project includes the following components:

- **Operating System:** Kali Linux
- **Virtualization Platform:** Oracle VM Virtual Machine
- **Penetration Testing Tool:** Burp Suite
- **Target Environment:** Vulnerable web application running in a controlled environment
- **Browser:** Mozilla, Firefox for web application interaction

This environment setup provides a safe and efficient platform for conducting web application penetration testing while ensuring that all testing activities remain within an isolated and authorized environment.

Network Configuration

The network configuration for this project was designed to allow proper communication between the browser, the penetration testing tool, and the target web application. This setup ensured that all web traffic could be intercepted and analyzed during the testing process.

Browser Proxy Configuration

To intercept web traffic, the web browser was configured to route its network requests through Burp Suite, which acts as an intercepting proxy. In this configuration, the browser sends all HTTP and HTTPS requests to Burp Suite before they reach the target web application.

The proxy settings were configured in the browser (such as Mozilla Firefox) by manually setting the proxy address to **127.0.0.1** and the port number to **8080**, **localhost:3000** which are the default proxy settings used by Burp Suite. This configuration allows Burp Suite to capture and display every request and response exchanged between the browser and the web application.

Once the proxy was enabled, the **Intercept** feature in Burp Suite allowed the tester to pause incoming requests, inspect their contents, and modify parameters if needed before forwarding them to the server. This capability is essential for performing security testing and identifying vulnerabilities within the application.

Target Environment Setup

For this project, the target web application used for testing was OWASP Juice-Shop, which is an intentionally vulnerable web application developed by the Open Worldwide Application Security Project (OWASP). It is designed specifically for learning and practicing web application security testing.

The OWASP Juice Shop application was deployed in the testing environment and accessed through the web browser running on the virtual machine. After the browser proxy configuration was completed, all interactions with the Juice Shop application passed through Burp Suite. This allowed the tester to monitor application requests, analyze parameters, and perform security testing activities.

By combining the proxy configuration with the vulnerable application environment, the project was able to simulate real-world penetration testing scenarios. This setup allowed different testing techniques to be performed, including analyzing authentication requests, manipulating input parameters, and identifying vulnerabilities within the application. Overall, the network configuration ensured that the browser, the testing tool, and the target application were properly connected, allowing an effective and controlled environment for conducting web application penetration testing.

Using Docker in Kali Linux to Run the Target Environment

Introduction to Docker

In this project, the target web application was deployed using Docker, a containerization platform that allows applications to run in isolated environments called containers. Containers package the application along with all its required dependencies, libraries, and configurations, ensuring that it runs consistently across different systems.

Docker is widely used in cybersecurity testing environments because it allows vulnerable applications to be deployed quickly without affecting the host operating system. By using containers, testers can easily start, stop, or remove the application after testing. This makes Docker an efficient tool for creating safe and controlled penetration testing environments.

Accessing the Kali Linux Terminal

The penetration testing process was performed using Kali Linux, which was installed inside a virtual machine. After starting the Kali Linux system, the terminal interface was opened to execute commands required for running the target application and security testing tools.

The terminal in Kali Linux provides a command-line interface that allows users to interact directly with the operating system. It enables users to install software, manage services, and run penetration testing tools through command-based operations.

To open the terminal in Kali Linux, the following steps were performed:

1. Start the Kali Linux virtual machine.
2. Log in to the system using the default credentials.
3. Open the terminal by selecting the terminal icon from the application menu or by using the keyboard shortcut **Ctrl + Alt + T**.

Once the terminal was opened, it was used to manage Docker containers and run the vulnerable web application.

Running the OWASP Juice Shop Using Docker

The vulnerable web application used in this project was OWASP Juice Shop, which is intentionally designed for learning web application security testing.

```

(mohini@kali)-[~]
$ docker --version
Docker version 28.5.2+dfsg3, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(mohini@kali)-[~]
$ sudo docker run --rm -p 3000:3000 bkimminich/juice-shop
[sudo] password for mohini:
info: Detected Node.js version v24.14.1 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file matching /*hacking-instructor-*.js$ is present (OK)
info: Port 3000 is available (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Server listening on port 3000
  
```

Fig: Successfully deployment of OWASP JUICE-SHOP container on port3000. (**docker run --rm -p 3000:3000 bkimminich/juice-shop**). This command performs the following operations: **docker run** – starts a new Docker container. **-d** – runs the container in detached mode (background). **-p 3000:3000** – maps port 3000 of the container to port 3000 of the host machine. **bkimminich/juice-shop** – downloads and runs the OWASP Juice Shop application image from the Docker repository

Advantages of Using Docker in the Project

Using Docker provided several advantages during the project:

- It allowed the vulnerable application to run in an isolated container environment.
- The application could be quickly deployed and removed when needed.
- The containerized environment ensured that the host system remained unaffected during testing.
- It simplified the setup process for penetration testing labs.

Accessing the Target Application

Once the container was successfully started, the application could be accessed through the web browser by navigating to the following address: **http://localhost:3000**

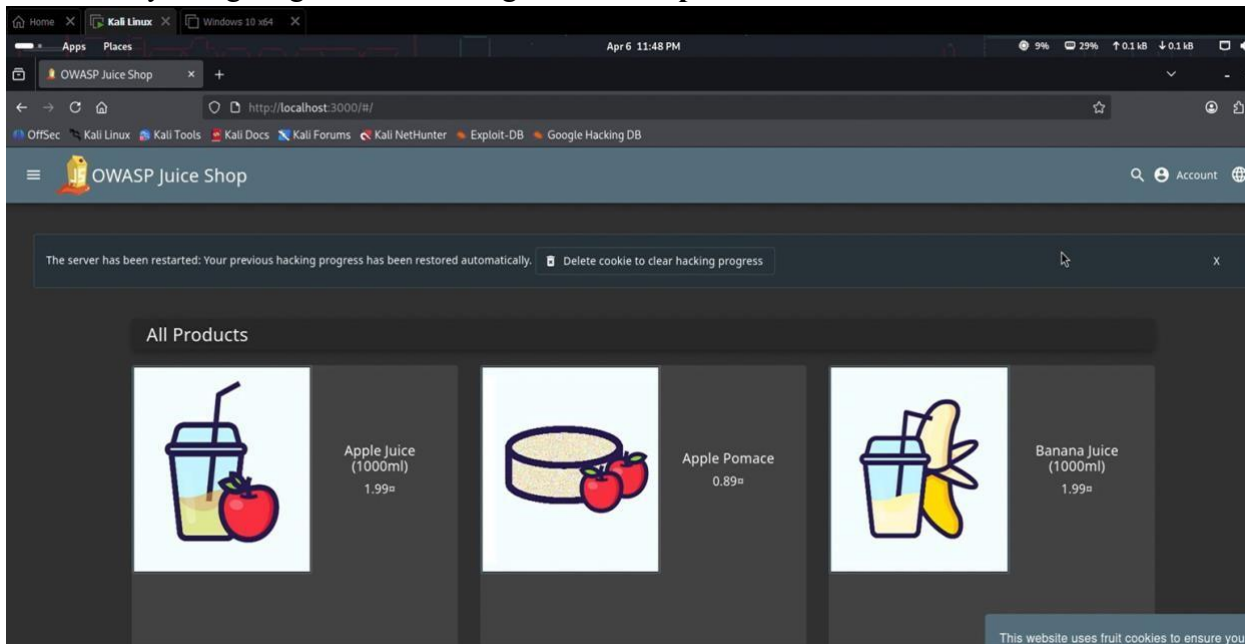


FIG shows the OWASP Juice Shop web interface, which served as the target environment for penetration testing activities.

Penetration Testing Implementation

This section describes the practical execution of the web application penetration testing process using **Burp Suite** on the target environment **OWASP Juice Shop**. The testing includes several attack techniques such as **brute force attacks**, **SQL injection testing**, and **cross-site scripting (XSS) analysis**. The objective of this phase is to identify vulnerabilities and verify whether they can be successfully exploited.

Burp Suite (Open):

After setting up the testing environment and launching the security tools, the next step was to start Burp Suite for web application penetration testing. When Burp Suite Professional is opened, the user is first presented with the project setup window.

In this stage, the **Temporary Project** option is selected, which allows the user to quickly start a testing session without saving the project configuration. After selecting the temporary project option, the **Next** button is clicked to proceed to the configuration stage, where the default settings are used.

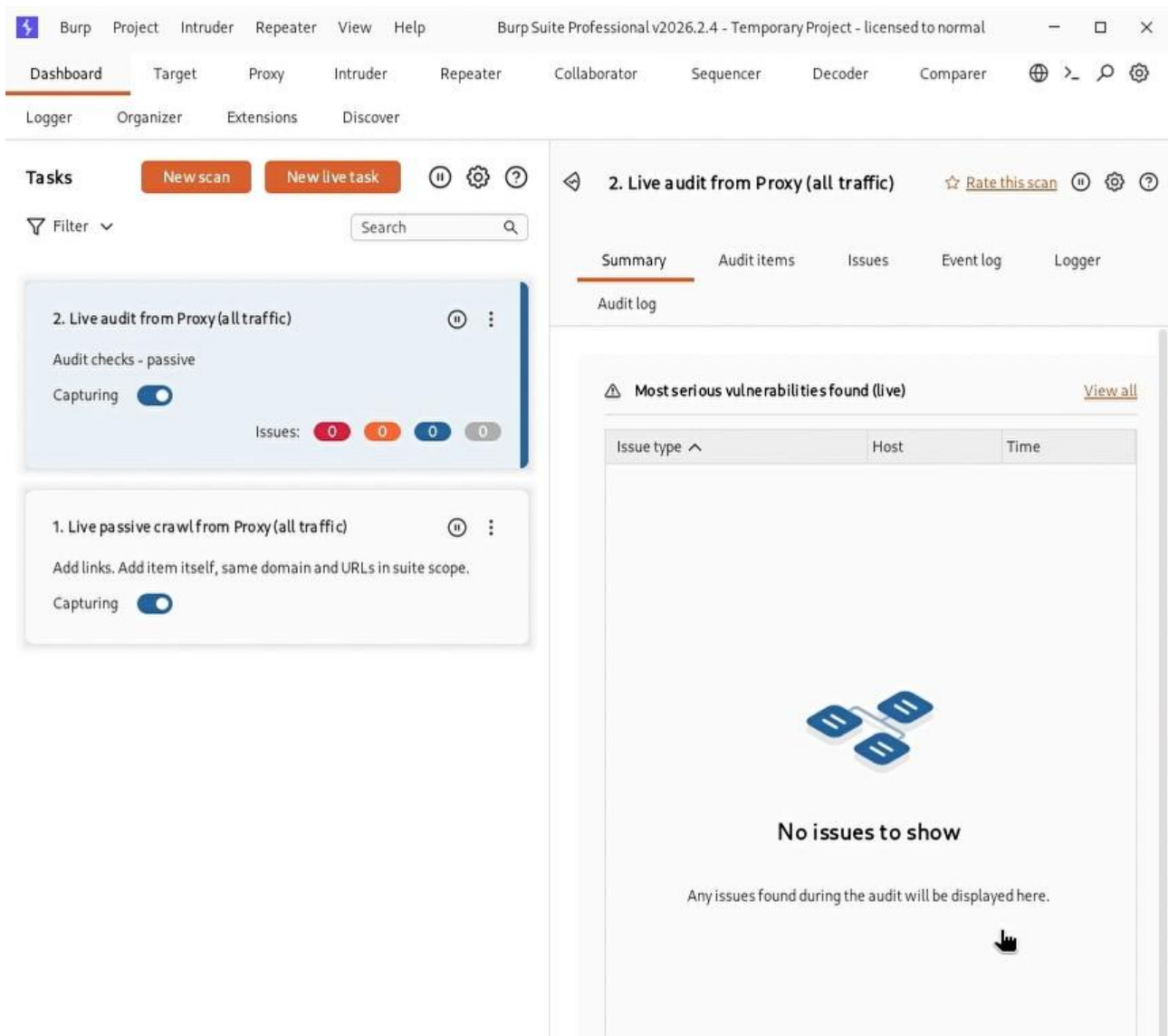
Once the configuration is confirmed, the **Start Burp** option is selected to launch the main interface of the application.

After Burp Suite starts successfully, the main dashboard of the tool becomes visible, displaying different tabs such as Dashboard, Target, Proxy, Intruder, Repeater, and others.

These components provide various functionalities required for web application security testing. To begin intercepting web traffic, the **Proxy** tab is selected from the main navigation panel. Within this section, Burp Suite provides the option to open a pre-configured browser that automatically routes all web traffic through the Burp proxy.

By clicking the **Open Browser** button, the built-in browser launches and is automatically configured to send all requests through Burp Suite.

This setup enables the tester to capture, inspect, and manipulate HTTP and HTTPS requests between the browser and the target web application, which is an essential step in performing web application penetration testing.



This figure shows the Burp Suite Professional: Burp Suite Dashboard with Live Proxy Traffic Monitoring and The **Live Audit from Proxy** feature is enabled, which allows the tool to monitor and analyze HTTP/HTTPS traffic captured through the proxy in real time.

Launch the Built-in Browser in Burp Suite

Click the **Open Browser** button in the Proxy section. This opens a pre-configured browser that automatically routes traffic through Burp Suite.

1. Access the Target Web Application

In the browser address bar, enter the URL of the target application OWASP Juice Shop.

2. Intercept Web Requests

As the application loads and user actions are performed, Burp Suite captures the HTTP/HTTPS requests sent between the browser and the web server.

3. Analyze and Modify Requests

Using the Proxy intercept feature, the tester can inspect, modify, and forward the captured requests to analyze the application's behaviour and identify vulnerabilities.

1.Brute Force Attack:

The brute force attack in this project was performed to test the strength of the login authentication mechanism of the target application OWASP Juice Shop using Burp Suite. The following steps describe the procedure used during the penetration testing process.

1. **Access the Login Page:**

Open the Burp Suite browser and navigate to the login page of the target application OWASP Juice Shop.

2. **Enable Proxy Interception:**

In Burp Suite, go to the **Proxy** tab and ensure that the **Intercept** option is turned on so that login requests can be captured.

3. **Enter Login Credentials:**

In the login form, enter a sample username and password and click the **Login** button.

4. **Capture the Login Request:**

Burp Suite intercepts the HTTP request sent by the login form. The request will appear in the Proxy **Intercept** window showing parameters such as email and password.

5. **Send Request to Intruder:**

Right-click on the captured login request and select **Send to Intruder**. This transfers the request to the Intruder tool for automated testing.

6. **Configure Attack Positions:**

In the **Intruder** tab, go to the **Positions** section. Select the password parameter and mark it as the target field where different values will be tested.

7. **Select Attack Type:**

Choose the **Sniper** attack type if only one parameter (such as the password field) needs to be tested.

8. **Load Password Wordlist:**

Go to the **Payloads** tab and load a password wordlist containing multiple possible passwords that will be tested automatically.

9. **Start the Attack:**

Click the **Start Attack** button. Burp Suite will send multiple login requests to the server using different password values from the wordlist.

10. **Analyze the Results:**

After the attack completes, analyze the server responses. Differences in response length, status codes, or messages may indicate a successful login attempt.

11. **Identify Valid Credentials:**

When a response shows a different pattern compared to other attempts, it may indicate the correct password for the account.

12. **Verify the Result:** Use the discovered credentials to log in to the application and confirm successful authentication. The result can also be verified by sending the login request through the **Repeater** tool in Burp Suite

2. SQL Injection:

SQL Injection is a web security vulnerability that allows attackers to interfere with the queries that an application makes to its database. It occurs when user input is not properly validated or sanitized before being included in SQL queries executed by the application. Attackers can exploit this vulnerability to bypass authentication mechanisms, access sensitive data, or modify and delete database records. SQL Injection attacks are commonly performed through input fields such as login forms, search boxes, or URL parameters. In penetration testing, SQL injection testing is performed to determine whether a web application is vulnerable to this type of attack.

In this project, SQL injection testing was performed on the target web application OWASP Juice Shop using Burp Suite. Burp Suite allowed the interception and modification of HTTP requests, enabling the tester to inject SQL payloads into the application's input fields and analyze the server responses.

Steps to Perform SQL Injection Using Burp Suite

- **Open the Target Application**

Launch the Burp Suite browser and navigate to the login page of the target application OWASP Juice Shop.

- **Enable Proxy Interception**

In Burp Suite, go to the **Proxy** tab and turn on the **Intercept** option to capture the login request.

- **Enter Test Credentials**

Enter a sample username and password in the login form and click the **Login** button.

- **Capture the Login Request**

Burp Suite intercepts the HTTP request sent to the server. This request contains parameters such as the email and password fields.

- **Send the Request to Repeater**

Right-click the captured request and select **Send to Repeater** to manually modify and test the request.

- **Insert SQL Injection Payload**

In the Repeater tab, modify the input field by inserting a SQL injection payload such as:

' OR 1=1 --

This payload attempts to manipulate the SQL query so that the authentication condition always evaluates as true.

- **Send the Modified Request**

Click the Send button in the Repeater tab to forward the modified request to the server.

- **Analyze the Server Response**

Examine the response returned by the server. If the login process bypasses authentication or behaves differently from normal responses, it may indicate that the application is vulnerable to SQL injection.

- **Verify the Result**

If the injected payload successfully bypasses authentication, it confirms that the application does not properly validate input and is vulnerable to SQL injection attacks.

During the penetration testing process, SQL injection testing is performed to evaluate the security of a web application's interaction with its backend database. This technique focuses on identifying weaknesses in input validation mechanisms that may allow attackers to manipulate database queries. By submitting specially crafted inputs through application forms or parameters, testers can observe whether the application improperly processes user input and executes unintended SQL commands. If such behaviour occurs, it may allow unauthorized access to sensitive data, authentication bypass, or manipulation of database records.

SQL injection testing was carried out on the intentionally vulnerable application OWASP Juice Shop using Burp Suite as the primary testing tool. Burp Suite was used to intercept and analyze HTTP requests exchanged between the browser and the web server. By modifying the parameters within these requests and injecting SQL payloads, it was possible to evaluate how the application handled malicious input. The responses returned by the server were carefully examined to determine whether the application was susceptible to SQL injection vulnerabilities.

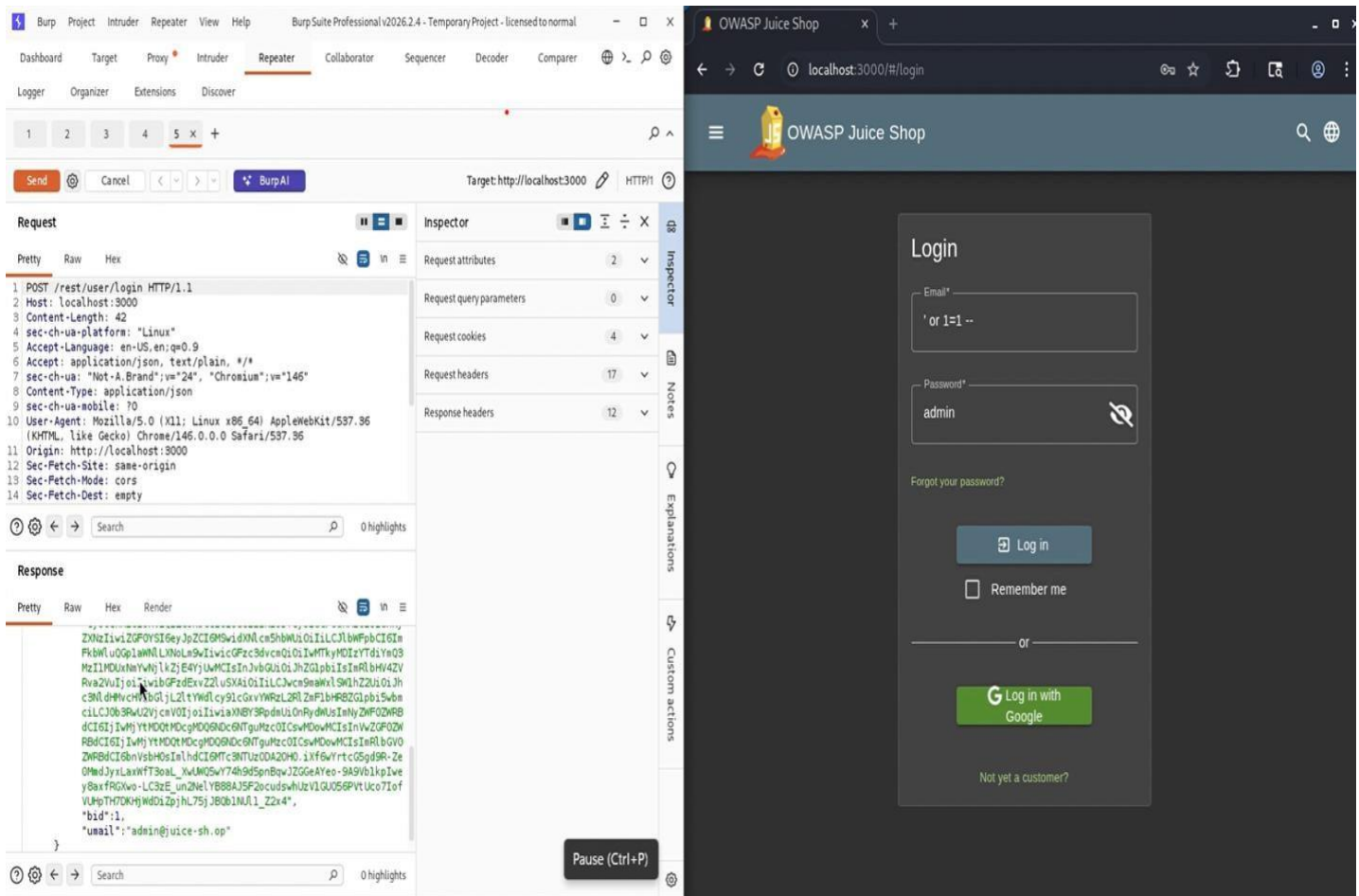


Fig: Using the SQL injection payload 'or 1=1— and resulting the “200 OK” response from the server. The SQL Injection to the OWASP Juice-Shop and also capture the authentication as appeared as in green colour. With help of SQL Injection, we don’t need the password. Password wouldn’t be right it will be anything we login as the administrator.

3. Cross-Site Scripting

(XSS) is a common web application security vulnerability that occurs when an application allows untrusted user input to be included in web pages without proper validation or sanitization. This vulnerability enables attackers to inject malicious scripts, usually written in JavaScript, into web pages that are viewed by other users. When the injected script is executed in a victim’s browser, it can perform unauthorized actions such as stealing session cookies, redirecting users to malicious websites, or modifying the content displayed on the page.

XSS vulnerabilities generally occur in areas where user input is accepted and later displayed back to the user without proper filtering. Because the malicious code runs within the victim’s browser, it can potentially compromise user data and affect the overall security of the application in this project, XSS testing was performed on the target web application OWASP Juice Shop using Burp Suite. The responses from the application were analyzed to determine whether the malicious scripts were executed, which would indicate the presence of a Cross-Site Scripting vulnerability. XSS vulnerabilities generally occur in areas where user input is accepted and later displayed back to the user without proper filtering.

Steps to Perform XSS Testing Using Burp Suite

1. Open the Burp Suite browser and navigate to the target application OWASP Juice Shop.
2. Enable the **Intercept** option in the **Proxy** tab of Burp Suite to capture requests.
3. Locate an input field in the application such as a search box, feedback form, or login field.

`<script>alert('XSS') </script>` (or) `<img src=x onerror=alert('XSS')>`

Submit the input and allow Burp Suite to intercept the request.

4. Send the captured request to the **Repeater** tab for manual testing and modification.
5. Inject the XSS payload into the selected parameter and send the modified request to the server.
6. Observe the response returned by the application and check whether the script is reflected and executed in the browser.
7. If the alert message or script executes in the browser, it indicates that the application is vulnerable to Cross-Site Scripting.

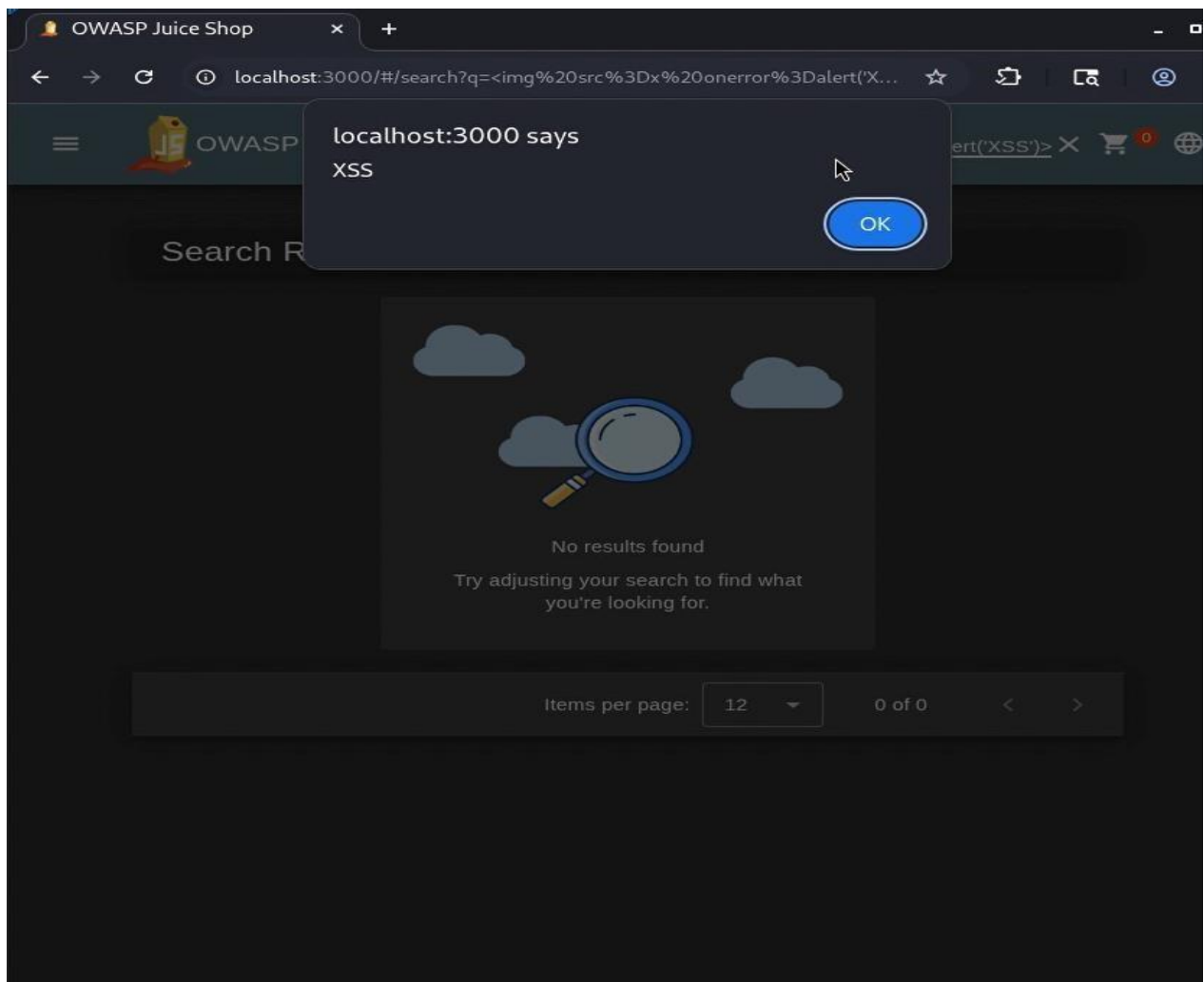


Fig shows the Cross-Site Scripting Vulnerability on the Web Application OWASP Juice-shop

Risks and Ethical Considerations

Risks Involved in Web Application Penetration Testing

Web application penetration testing is an important process for identifying security vulnerabilities; however, it also carries certain risks if not conducted in a controlled environment. During testing activities such as brute force attacks, SQL Injection, and Cross-Site Scripting, there is a possibility that the application or system may behave unexpectedly. For example, poorly handled attacks could cause system crashes, data corruption, or temporary service disruptions.

Another risk involves unauthorized testing. Performing penetration testing on a system without proper permission is illegal and may violate cybersecurity laws and organizational policies.

Additionally, if sensitive data is exposed during testing, it must be handled carefully to prevent misuse or data leakage. To reduce these risks, this project was conducted in a **controlled laboratory environment** using intentionally vulnerable applications such as OWASP Juice Shop. This ensured that the testing activities did not affect real users, production systems, or confidential data.

Legal Risks

Performing penetration testing without proper authorization from the system owner may violate cybersecurity laws and regulations. Unauthorised access to computer systems or networks can lead to legal consequences, including penalties or criminal charges. Accessing, modifying, or extracting sensitive information during testing without permission may breach data protection laws. Security testing activities such as brute force attempts or injection attacks could be considered illegal if conducted on real production systems without approval.

Ethical Risks

- Penetration testing techniques must not be used for malicious purposes or personal gain.
- Security testers have a responsibility to protect any sensitive data discovered during testing.
- Vulnerabilities identified during testing should be responsibly reported to the appropriate authorities or developers.
- Ethical hackers must ensure that testing activities do not intentionally harm systems, users, or organisations.

Ethical Considerations

Ethics play a critical role in cybersecurity and penetration testing. Security testing techniques can potentially be misused if applied with malicious intent. Therefore, penetration testers must follow ethical guidelines and professional standards while conducting security assessments.

In this project, testing was performed strictly for **educational and research purposes** to understand how vulnerabilities occur in web applications and how they can be identified and mitigated. All testing activities were conducted using authorized tools such as Burp Suite in a safe and isolated environment. Ethical penetration testing focuses on identifying weaknesses so that developers and organizations can improve the security of their systems.

The ultimate goal is to strengthen application security, protect user data, and prevent potential cyberattacks. All testing activities in this project were performed in a controlled environment using intentionally vulnerable applications such as OWASP Juice Shop and testing tools like Burp Suite

Table of Discovered Vulnerabilities

Vulnerability	Description	Risk Level	Recommended Technical Fix
SQL Injection	The application allows user input to be directly included in SQL queries without proper validation. Attackers can manipulate database queries to bypass authentication or access sensitive data.	High	Use prepared statements and parameterized queries to separate SQL code from user input. Implement input validation and use ORM frameworks to prevent direct query manipulation.
Cross-Site Scripting (XSS)	The application does not properly sanitize user input before displaying it on web pages. This allows attackers to inject malicious scripts that execute in the victim's browser.	High	Implement input validation and output encoding . Use security libraries to sanitize user input and Content Security Policy (CSP) to prevent execution of unauthorized scripts.
Weak Authentication Mechanism	The login system is vulnerable to brute force attacks because it allows multiple login attempts without restrictions.	Medium	Implement account lockout mechanisms , rate limiting, and CAPTCHA verification after multiple failed login attempts.
Improper Session Management	Session tokens may not be properly protected, which could allow attackers to hijack user sessions.	Medium	Use secure session management techniques , enable HTTPS encryption , set secure cookie flags such as HTTP Only and Secure, and implement session expiration policies.
Sensitive Data Exposure	Sensitive data transmitted between the client and server may be exposed if proper encryption mechanisms are not enforced.	Medium	Enforce HTTPS using SSL/TLS encryption and ensure that sensitive information is not stored or transmitted in plain text.

During the penetration testing process, several vulnerabilities were identified in the target web application OWASP Juice Shop using Burp Suite. These vulnerabilities could potentially allow attackers to bypass authentication mechanisms, inject malicious scripts, or gain unauthorized access to sensitive information. To improve the security of the application, it is important to apply proper remediation techniques and follow secure development practices.

The following section presents the discovered vulnerabilities along with their associated risks and recommended technical solutions.

Input Validation and Sanitization

All user inputs should be validated on both the client side and server side. Proper input filtering helps prevent attacks such as SQL Injection and Cross-Site Scripting by ensuring that only expected data formats are accepted.

Secure Authentication Mechanisms

The application should enforce strong authentication policies, including strong password requirements, multi-factor authentication (MFA), and login attempt limitations to prevent brute force attacks.

Regular Security Testing

Organizations should perform periodic vulnerability assessments and penetration testing using security tools such as Burp Suite. Continuous security testing helps identify vulnerabilities before attackers can exploit them.

Implement Security Headers

Security headers such as **Content Security Policy (CSP)**, **X-Frame-Options**, and **X-Content-TypeOptions** should be configured to enhance browser-level protection against common web attacks.

Security Awareness and Monitoring

System administrators should continuously monitor application logs and security events to detect suspicious activities. Proper security awareness training should also be provided to developers and system administrators.

Conclusion

This project focused on the development and execution of a web application penetration testing framework using Burp Suite to identify common security vulnerabilities in modern web applications. As web applications continue to play a major role in areas such as online banking, e-commerce, education, and communication, ensuring their security has become a critical requirement. The objective of this project was to understand how penetration testing tools can be used to analyze web application behaviour, detect vulnerabilities, and demonstrate potential exploitation techniques in a controlled environment.

During the testing process, several common web vulnerabilities were examined using the intentionally vulnerable application OWASP Juice Shop. The framework involved intercepting and analyzing HTTP requests, modifying parameters, and testing how the application responded to manipulated inputs. Using the features available in Burp Suite, multiple testing techniques were applied to evaluate the security posture of the application.

The penetration testing process successfully demonstrated several attack scenarios, including brute force attempts against authentication mechanisms, SQL Injection testing to manipulate database queries, and Cross-Site Scripting (XSS) testing to evaluate the application's handling of user-generated input. Through these tests, it was possible to identify weaknesses in input validation, authentication controls, and application response behaviour. The project also demonstrated how captured requests could be analyzed and modified using tools such as the Proxy, Intruder, and Repeater modules in Burp Suite.

The findings from the penetration testing process highlight the importance of implementing secure coding practices and regular security assessments in web application development. Many vulnerabilities arise from insufficient input validation, weak authentication mechanisms, and poor session management. By identifying these weaknesses during the testing phase, developers and security professionals can implement appropriate mitigation strategies to improve the security of the application.

Overall, the project successfully demonstrated how a structured penetration testing approach can be applied to analyze web application security. The use of Burp Suite as a testing framework provided valuable insights into how attackers might exploit vulnerabilities and how organizations can strengthen their defences against such threats.

Future Scope

Although the framework developed in this project successfully demonstrated the identification of several web application vulnerabilities, there are several opportunities for further improvement and expansion. Future work can focus on enhancing the automation, scalability, and analytical capabilities of the penetration testing framework.

One possible improvement is the integration of advanced extensions available within Burp Suite. Extensions available through the Burp Suite extension marketplace can significantly enhance the functionality of the testing environment.

For example, tools such as Logger++ can be integrated to provide detailed logging and monitoring of HTTP requests and responses. This extension allows testers to capture and analyze large volumes of traffic more efficiently, which improves the ability to identify unusual or suspicious patterns during testing.

Another useful extension is Authorize, which can be used to test authorization vulnerabilities. This extension helps security testers evaluate whether users with different permission levels can access restricted resources.

Future improvements can also include integrating additional automated vulnerability scanning tools and scripting capabilities to improve testing efficiency. Automation frameworks can help reduce manual testing time and allow security teams to perform regular vulnerability assessments as part of the development lifecycle.

Another potential area of enhancement is the integration of penetration testing with **continuous security testing practices**, such as DevSecOps.

Furthermore, future research can explore testing more advanced web application vulnerabilities such as business logic flaws, API security weaknesses, and authentication bypass techniques. Expanding the testing framework to include API testing, mobile application security analysis, and cloud-based application security would also increase its practical value.

In conclusion, while this project successfully demonstrated the use of Burp Suite for web application penetration testing, the framework can be further improved through the integration of advanced extensions, automated testing techniques, and continuous security monitoring practices. These improvements will help create a more comprehensive and efficient security testing environment capable of identifying a wider range of vulnerabilities in modern web applications.

References

The following sources were used to understand web application penetration testing concepts, tools, and methodologies used in this project.

1. OWASP. *OWASP Web Security Testing Guide*. Available at: <https://owasp.org>
 - Provides comprehensive guidance on web application security testing methodologies and common vulnerabilities.
2. PortSwigger. *Burp Suite Documentation*. Available at: <https://portswigger.net>
 - Official documentation explaining the usage, features, and configuration of Burp Suite.
3. Kali Linux Documentation. Available at: <https://www.kali.org/docs/>
 - Official documentation for the Kali Linux penetration testing platform and its tools.
4. OWASP Juice Shop Project Documentation. Available at: <https://owasp.org/www-projectjuice-shop/>
 - Documentation describing the intentionally vulnerable web application used for security testing and learning.
5. OWASP Foundation. *OWASP Top 10 Web Application Security Risks*.
 - A widely recognized list of the most critical security risks affecting web applications.

Appendix

The appendix section provides additional technical details, logs, and configuration information that support the penetration testing activities performed in this project.

Docker Command for Running the Target Application: `docker run --rm -p 3000:3000 bkimminich/juice-shop`

HTTP Request Captured in Burp Suite

POST /rest/user/login HTTP/1.1

Host: localhost:3000

User-Agent: Mozilla/5.0

Content-Type: application/Json

Content-Length: 45

```
{
  "email": "admin@juice-sh.op",
  "password": "test123"
}
```

SQL Injection Test Payload: ' OR 1=1 --

Cross-Site Scripting Payload: `<script>alert('XSS') </script>` or

`<img src=x onerror=alert('XSS')>`

